

Deploying Your First Project

Overview

This comprehensive guide walks you through deploying your StudAI-generated applications to popular hosting platforms. Deploy your projects with confidence using our step-by-step instructions.

Supported Deployment Platforms

Vercel (Recommended for Frontend)

Best for: React, Vue, Next.js, static sites

Deployment time: ~2 minutes

Prerequisites

- GitHub, GitLab, or Bitbucket account
- Project pushed to your repository

Step-by-Step Deployment

1. Connect Your Repository

- Visit vercel.com and sign in
- Click **"New Project"**
- Import your StudAI project repository

2. Configure Build Settings

```
# Build Command (auto-detected)
npm run build

# Output Directory
dist/ or build/

# Install Command
npm install
```

3. Environment Variables

- Add API keys and configuration variables
- Use Vercel's environment variable section
- Separate staging and production configs

4. Deploy

- Click **"Deploy"**

- Monitor build logs for any issues
- Your app will be live in ~60 seconds

Custom Domain Setup

```
# Add custom domain in Vercel dashboard
# Configure DNS records:
# Type: CNAME, Name: www, Value: cname.vercel-dns.com
# Type: A, Name: @, Value: 76.76.19.61
```

Netlify (Great for Static Sites)

Best for: Static sites, JAMstack applications

Deployment time: ~3 minutes

Deployment Methods

Method 1: Git Integration

1. Connect your Git repository
2. Set build command: `npm run build`
3. Set publish directory: `dist` or `build`
4. Deploy automatically on every push

Method 2: Drag & Drop

1. Build your project locally: `npm run build`
2. Drag the build folder to Netlify
3. Instant deployment

Netlify Features

- **Form Handling:** Built-in form processing
- **Edge Functions:** Serverless functions at the edge
- **Split Testing:** A/B testing capabilities

Railway (Full-Stack Applications)

Best for: Node.js, Python, full-stack apps

Deployment time: ~5 minutes

Setup Process

1. Connect Repository

- Link your GitHub repository
- Railway auto-detects your framework

2. Environment Configuration

```
# Add environment variables
DATABASE_URL=postgresql://...
API_KEY=your_api_key_here
PORT=3000
```

3. Database Setup

- Add PostgreSQL, MySQL, or Redis
- Automatic connection string generation
- Built-in database management tools

4. Deploy

- Automatic deployments on git push
- View logs in real-time
- Custom domain support

Pre-Deployment Checklist

Code Optimization

- ☐ Remove console.log statements
- ☐ Optimize images and assets
- ☐ Minify CSS and JavaScript
- ☐ Enable gzip compression

Environment Setup

- ☐ Configure environment variables
- ☐ Set up database connections
- ☐ Test API endpoints
- ☐ Verify CORS settings

Security Measures

- ☐ Update dependencies
- ☐ Configure HTTPS
- ☐ Set up proper authentication
- ☐ Implement rate limiting

Common Deployment Issues

Build Failures

Problem: Build process fails during deployment

Solutions:

```
# Check Node.js version compatibility
"engines": {
  "node": ">=16.0.0"
}

# Clear cache and reinstall dependencies
npm ci --legacy-peer-deps

# Check for missing environment variables
console.log(process.env.NODE_ENV)
```

API Connection Issues

Problem: Frontend can't connect to API

Solutions:

- Update API URLs for production
- Configure CORS headers
- Check environment variable names

Performance Issues

Problem: Slow loading times

Solutions:

- Enable CDN
- Optimize bundle size
- Implement lazy loading
- Use image optimization

Advanced Deployment Strategies

Multi-Environment Setup

```
# Example: Different configs for staging/production
staging:
  api_url: https://api-staging.yourapp.com
  database: staging_db

production:
```

```
api_url: https://api.yourapp.com
database: production_db
```

CI/CD Pipeline Integration

- Automated testing before deployment
- Staged rollouts
- Rollback capabilities
- Performance monitoring

Database Migration

```
# Example migration commands
npm run migrate:up           # Apply migrations
npm run migrate:down        # Rollback migrations
npm run seed:run             # Populate initial data
```

Monitoring and Maintenance

Essential Monitoring Tools

- **Uptime Monitoring:** UptimeRobot, Pingdom
- **Error Tracking:** Sentry, Bugsnag
- **Performance:** Google PageSpeed, GTmetrix
- **Analytics:** Google Analytics, Mixpanel

Regular Maintenance Tasks

- Update dependencies monthly
- Monitor performance metrics
- Review and rotate API keys
- Backup database regularly

Troubleshooting Guide

Connection Timeouts

```
# Increase timeout values
server.timeout = 120000; // 2 minutes

# Check network connectivity
curl -I https://your-api.com/health
```

Memory Issues

```
# Increase Node.js memory limit
node --max-old-space-size=4096 server.js

# Monitor memory usage
process.memoryUsage()
```

SSL Certificate Issues

- Verify domain ownership
- Check certificate expiration
- Force HTTPS redirects

Cost Optimization

Free Tier Limits

- **Vercel:** 100GB bandwidth/month
- **Netlify:** 100GB bandwidth/month
- **Railway:** \$5 credit monthly

Scaling Considerations

- Monitor usage metrics
- Implement caching strategies
- Optimize database queries
- Use CDN for static assets

Estimated completion time: 8 minutes for basic deployment, additional time for advanced configurations.